

UNIVERSIDAD AUTÓNOMA DE CHIAPAS

Facultad de Ciencias en Física y Matemáticas · IDTS
Ingeniería en Desarrollo y Tecnologías de Software

MedIA — Expediente Clínico Electrónico

Distrito de Salud I · Tuxtla Gutiérrez, Chiapas

DOCUMENTO 1 — REVISIÓN 2

División de Trabajo Rebalanceada

Con referencias cruzadas a documentos técnicos

Semestre	8° Semestre IDTS — UNACH, Enero–Junio 2026
Materia	Cómputo Distribuido · Grupos 8M / 8N / 8O
Versión	2.0 — Rebalanceo de cargas · Marzo 2026
Integrantes	5 personas · Roles ajustados por experiencia y disponibilidad

1. Criterios del Rebalanceo

La revisión de la división original detectó que el Lead asumió simultáneamente la totalidad del trabajo de arquitecto, DBA, PM y desarrollador. Los documentos técnicos (Doc1–Doc7) ya están entregados y representan trabajo de diseño significativo. El rebalanceo redistribuye la carga de implementación con base en tres criterios:

- **Equidad entre implementadores:** Personas 3, 4 y 5 asumen cargas equivalentes en términos de módulos, endpoints y semanas activas.
- **Reconocimiento del trabajo previo del Lead:** El Lead ya diseñó el sistema completo. En la fase de implementación actúa principalmente como coordinador técnico y revisor.
- **Continuidad de la base frontend:** Persona 2 establece la cimentación del frontend y los seeds, trabajo crítico y bien documentado en Doc5 y Doc7 del que depende todo el equipo desde la primera semana.

Nota importante sobre referencias cruzadas

Este documento usa referencias explícitas a Doc2–Doc7 en cada tarea. El formato es [DocX §Y] donde X es el número de documento e Y la sección. Esto permite que cualquier integrante pueda dar el documento a un LLM (como ChatGPT o Claude) junto con el documento referenciado y obtener código funcional alineado con las decisiones de arquitectura ya tomadas.

2. Distribución General — Tabla Resumen

#	Persona	Rol	Semanas activas	Carga
P1	Lead / Coordinador	Arquitectura, revisión, deploy	S1 (setup) + S8 (deploy)	Mínima
P2	Frontend Base & Seeds	Cimentación frontend + catálogos BD	S1–S2 (+ revisiones S5)	Alta
P3	Backend Clínico Senior	Pacientes, expediente, referencias	S3–S4, S6	Alta
P4	Backend SOAP & Consulta	Encuentros, SOAP, firma, PDF nota	S4–S5	Alta
P5	Backend Prescripciones & Admin	Prescripciones, lab, auditoría, admin	S5–S7	Alta

3. División de Trabajo por Persona

3.1 — Persona 1 — Lead · Coordinador Técnico & Deploy

Semanas activas	S1 (setup inicial, 2–3 días) + S8 (deploy Azure, 2 días) + revisión de PRs continua
Carga estimada	Mínima — ya entregó la totalidad del diseño del sistema

Rol en el proyecto

El Lead entregó los 7 documentos técnicos del proyecto (arquitectura, modelo BD 40 entidades, API REST completa, infraestructura Azure, seeds, diagramas y UI/UX). Ese trabajo equivale a la carga de un sprint completo de diseño. Durante la implementación actúa como coordinador: desbloquea al equipo, aprueba PRs y configura el deploy final. No se le asigna desarrollo de módulos nuevos.

Setup inicial del repositorio (S1 — 2 días)

- Crear repositorio en GitHub Organization unach-media-ece con estructura de carpetas documentada en Doc4 §2.
- Subir docker-compose.yml de desarrollo tal como está especificado en Doc4 §2.1 (ya redactado, solo copiar y validar).
- Crear .env.example con todas las variables listadas en Doc4 §1.1 (tabla ya completa).
- Crear scripts start_dev.sh, migrate.sh y seed.sh — scripts de una línea que ejecutan docker compose up, alembic upgrade head y python seeds/run_seeds.py respectivamente.
- Configurar GitHub branch protection: rama main requiere PR aprobado por P1, rama develop acepta PRs de todos.

 **Referencia:** Docker compose y variables de entorno: Doc4 §2.1 y §1.1 — copiar directamente, están completos.

Revisión técnica continua (S1–S8 — sin código)

- Revisar y aprobar Pull Requests al branch develop (validar que siguen convenciones de Doc1 §5 y §6).
- Responder dudas técnicas del equipo sobre la arquitectura de Doc2 y los endpoints de Doc3.
- Mantener NOTAS_PENDIENTES.md actualizado con bloqueantes detectados en revisiones.

 **Referencia:** Convenciones de commits y PR: Doc1 §5 — Convenciones del Repositorio Git.

 **Referencia:** Arquitectura para resolver dudas: Doc2 §1 — Stack Tecnológico y Justificación.

Deploy a Azure (S8 — 2 días, configuración, no código)

- Configurar Azure App Service con variables de entorno de producción (Doc4 §1.1, columna Producción).
- Configurar Azure Static Web Apps con deploy automático desde rama main.

- Configurar Azure Front Door + CDN con las reglas descritas en Doc6 §Diagrama 1.
- Cargar secretos en Azure Key Vault usando Managed Identity (especificado en Doc6 §Capa 5).
- Validar HTTPS, CORS (CORS_ORIGINS=https://media-chiapas.azurestaticapps.net) y health check.

 **Referencia:** *Infraestructura y deploy: Doc4 §3 y §4 — procedimiento completo con comandos az CLI.*

 **Referencia:** *Arquitectura de producción: Doc6 §Diagrama 1, Capa 3, 4 y 5 — conexiones y protocolos.*

3.2 — Persona 2 — Frontend Base & Seeds — Cimentación del Sistema

Semanas activas	S1–S2 (frontend base + seeds) + S5 (ajustes menores)
Carga estimada	Alta — la base del sistema sobre la que el resto del equipo trabaja desde el primer día

Rol en el proyecto

Persona 2 establece la base que el resto del equipo necesita: el layout que todos van a ver, el sistema de autenticación en el frontend, los componentes compartidos y los seeds que hacen que la BD tenga datos desde el primer día. Sin este trabajo, P3, P4 y P5 no tienen entorno funcional para desarrollar. Las especificaciones de Doc7 (UI/UX) y Doc5 (seeds) sirven de referencia para garantizar coherencia, pero la implementación requiere criterio técnico y coordinación estrecha con P1.

Seeds SQL — Catálogos de la BD (S1, primeros 3 días)

- Ejecutar el script `convert_localidades.py` de Doc5 §2.3 para generar el INSERT de ~5,000 localidades de Chiapas desde el archivo `ITER_07XLSX10.xlsx` del INEGI.
- Crear `02_seeds_geograficos.sql` con los 32 estados y 124 municipios de Chiapas — fragmentos SQL completos en Doc5 §2.1 y §2.2, solo completar los registros faltantes.
- Crear `03_seeds_cie10.sql` con los ~14,400 códigos CIE-10 descargados desde la DGIS (URL en Doc5 §3.1). Usar el mismo patrón de `INSERT ON CONFLICT DO NOTHING`.
- Crear `04_seeds_medicamentos.sql` con los ~900 medicamentos del Cuadro Básico SSA (Doc5 §3.2).
- Crear `05_seeds_catalogos.sql` con: `cat_lenguas_indigenas` (Doc5 §3.3), `cat_especialidades_medicas`, los 6 roles, los 8 módulos del sistema y la matriz de ~192 `permisos_rol` (Doc5 §4).
- Crear `06_seeds_superadmin.sql`: INSERT del usuario `superadmin@media.local` con contraseña hashada con Argon2id (Doc5 §5 — se coordina con P1 para el hash inicial).

 **Referencia:** SQL completo de estados y municipios: Doc5 §2.1 y §2.2 — fragmentos listos.

 **Referencia:** Script Python para localidades: Doc5 §2.3 — código completo, solo ejecutar.

 **Referencia:** Seeds de roles y permisos: Doc5 §4 — matriz CRUD completa.

Setup del frontend (S1, últimos 2 días)

- Crear proyecto con: `npm create vite@latest frontend -- --template react`. Instalar dependencias: `react-router-dom`, `axios`, `shadcn/ui`, `tailwindcss`.
- Configurar `tailwind.config.js` con la paleta de colores de Doc7 §1.1: fondo `#EDEBE6`, azul institucional `#1B4F8A`, sidebar `#101E33`.
- Configurar `vite.config.js` con proxy de desarrollo hacia `http://localhost:8000` para evitar CORS en local.
- Agregar import de DM Sans en `index.html` tal como especifica Doc7 §2 (link de Google Fonts).
- Crear la estructura de carpetas: `src/pages/`, `src/components/`, `src/api/`, `src/hooks/`, `src/utils/`, `src/context/` (nomenclatura de Doc1 §6.2).

 **Referencia:** Paleta de colores exacta: Doc7 §1.1 y §1.2 — hex codes completos para Tailwind.

 **Referencia:** Convenciones de carpetas y nombres: Doc1 §6.2 — nomenclatura React acordada.

Layout base: Sidebar + TopBar (S1–S2)

- Implementar Sidebar.jsx con fondo #101E33, ancho 220px (colapsable a 56px en <1280px), grupos CLÍNICA y SISTEMA.
- Grupo CLÍNICA: Dashboard, Pacientes, Expediente, Consulta, Referencias, Documentos. Grupo SISTEMA: Auditoría (AUDITOR+), Administración (ADMIN+), Seguridad (SUPERADMIN).
- Footer del sidebar: Avatar + nombre del usuario + badge de rol + indicador de conexión. El rol siempre visible (Doc7 §3.1 — NO CAMBIAR).
- Item activo: fondo #162540 + borde izquierdo 3px #1B4F8A. Hover: #162540 (Doc7 §1.2).
- Implementar TopBar.jsx con altura 56px, fondo #FFFFFF, border-bottom #DAD4CC. Zona derecha: notificaciones + avatar + botón logout.
- El layout envuelve a todas las páginas mediante ProtectedRoute — solo visible con JWT válido.

 **Referencia:** Especificaciones exactas de Sidebar y TopBar: Doc7 §3.1 — dimensiones, colores y contenido.

 **Referencia:** Colores de sidebar: Doc7 §1.2 — tokens #101E33, #162540, #F5F2EC.

 **Referencia:** Arquitectura del frontend: Doc6 §Diagrama 1, Capa 2 — 9 módulos y roles del sidebar.

AuthContext + Login 2FA (S2)

- Implementar AuthContext.jsx con useState para el access_token (NUNCA localStorage, Doc1 §6.2 Reglas de Seguridad). El contexto expone: user, token, login(), logout(), refreshToken().
- Implementar refresh silencioso: useEffect que calcula cuándo expira el token (exp del JWT decodificado) y llama a POST /auth/refresh 60 segundos antes, sin interrumpir al usuario.
- Implementar LoginPage.jsx con flujo de 2 pasos: (1) formulario usuario+contraseña → si requiere_2fa=true, (2) input de 6 dígitos OTP individual por carácter.
- Conectar con POST /auth/login y POST /auth/2fa/verify según contratos de Doc3 §Módulo 1.
- Implementar ProtectedRoute.jsx que redirige a /login si no hay token, y a /403 si el rol no tiene permiso.

 **Referencia:** Flujo de login con 2FA paso a paso: Doc6 §Diagrama 4 — precondiciones, pasos y estados.

 **Referencia:** Endpoints de autenticación: Doc3 §Módulo 1 — request/response completos.

 **Referencia:** JWT en memoria (seguridad): Doc2 §1.1 — justificación técnica contra XSS.

Componentes shadcn/ui compartidos (S2)

- Instalar e inicializar shadcn/ui: npx shadcn@latest init. Copiar componentes: Button, Input, Badge, Card, Table, Dialog (Modal), Spinner, Alert.
- Crear Badge.jsx con variantes de color para estados del sistema (Doc7 §1.3): **alergia CRÍTICA** → #DC2626, **alergia MODERADA** → #D97706, **nota FIRMADA** → #2D8653, **encuentro ABIERTO** → #1B4F8A.
- Crear DashboardPage.jsx condicional según rol: para MEDICO_GENERAL muestra lista de pacientes pendientes; ENFERMERIA muestra cola de signos vitales;

ADMINISTRADOR muestra accesos directos a CRUD; AUDITOR muestra últimos incidentes.

 **Referencia:** Colores semánticos de badges y alertas: Doc7 §1.3 — tabla completa de contextos y colores.

 **Referencia:** Roles del sistema: Doc6 §Diagrama 2 — casos de uso por actor.

Ajustes menores de frontend (S5 — 1–2 días)

- Incorporar ajustes de UX detectados en S3–S4: asegurarse de que el número de expediente aparece en el TopBar desde que se abre un encuentro (Doc6 §Diagrama 3, Precondición).
- Revisar que los componentes Badge aplican los colores correctos de Doc7 §1.3 en la vista de alergias.

3.3 — Persona 3 — Backend Clínico Senior

Semanas activas	S3–S4 (backend) + S6 (referencias + frontend expediente)
Carga estimada	Alta — núcleo clínico del sistema

Rol en el proyecto

Persona 3 implementa el corazón del sistema: el expediente del paciente y el sistema de referencias médicas entre establecimientos. Cubre las 4 Reglas de Negocio fundamentales que garantizan el cumplimiento de la NOM-004. Su trabajo en S3–S4 es la base sobre la que P4 construye el módulo de consulta.

Módulo Personas + Catálogos geográficos (S3, días 1–3)

- Implementar POST /personas, GET /personas, GET /personas/{id}, PATCH /personas/{id} según contratos de Doc3 §Módulo 2.
- CURP es NULL permitido (decisión de inclusión para comunidades indígenas de Chiapas — no modificar, Doc3 §Módulo 2 Consideraciones).
- Implementar GET /catalogos/estados, GET /catalogos/municipios?estado={clave}, GET /catalogos/localidades?municipio={clave}, GET /catalogos/lenguas — estos endpoints leen de los seeds cargados por P2 y se configuran con TTL de 24h en CDN (Doc6 §Diagrama 1, Capa 3).
- El campo id_lengua_materna debe activar una alerta visual de barrera lingüística — el backend incluye el flag en el response, el frontend de P4 la muestra en el módulo de encuentros (Doc3 §Módulo 2).

 **Referencia:** Contratos de endpoints de Personas: Doc3 §Módulo 2 — todos los campos, validaciones y códigos HTTP.

 **Referencia:** Justificación CURP NULL: Doc3 §Módulo 2 — Consideraciones normativas.

Módulo Pacientes + Tutores (S3, días 4–5)

- Implementar POST /pacientes, GET /pacientes, GET /pacientes/{id}, GET /pacientes/{id}/expediente según Doc3 §Módulo 3.
- El número_expediente se genera automáticamente con formato EXP-YYYY-{secuencia} al crear el paciente — usar SEQUENCE de PostgreSQL (Doc3 §Módulo 3, Reglas de negocio).
- GET /pacientes/{id}/expediente aplica las Reglas 1, 2 y 3: solo muestra antecedentes si hay encuentro activo o referencia ACEPTADA (implementar en el service, no en el router).
- POST /pacientes/{id}/tutores requiere que el documento_legal ya esté en Azure Blob Storage (P5 implementa el upload; P3 solo guarda la URL).

 **Referencia:** Endpoints y reglas de pacientes: Doc3 §Módulo 3 — CRUD completo con reglas de acceso.

 **Referencia:** Flujo de acceso cross-establecimiento: Doc6 §Diagrama 5, Sub-flujo B — Reglas 2 y 3 paso a paso.

Módulo Expediente — Antecedentes, Alergias e Inmunizaciones (S4, días 1–3)

- Implementar CRUD completo de alergias con soft delete forense: DELETE físico prohibido, solo borrado lógico con motivo_baja obligatorio (Doc3 §Módulo 4, Req 6 Forense).
- Severidad de alergias: LEVE / MODERADA / CRÍTICA — el backend incluye este campo en todos los responses para que el frontend P4 muestre el badge correcto del sistema de colores de Doc7 §1.3.
- Implementar CRUD de los 4 tipos de antecedentes: heredofamiliares, patológicos, no patológicos y ginecoobstétricos. Para ginecoobstétricos, validar en el service que el sexo del paciente sea 'F' antes de permitir la creación.
- Implementar CRUD de inmunizaciones con soft delete y trazabilidad en historial_cambios.
- Implementar vistas SQL en PostgreSQL: v_paciente_basico (datos mínimos de acceso universal), v_signos_encuentro (últimos signos vitales por encuentro) y v_auditoria_estadistica (estadísticas agregadas para el panel de P5).

 **Referencia:** Endpoints de alergias y antecedentes: Doc3 §Módulo 4 — tablas afectadas y reglas de soft delete.

 **Referencia:** Colores de badge por severidad de alergia: Doc7 §1.3 — #DC2626 CRÍTICA, #D97706 MODERADA.

Las 4 Reglas de Negocio en services/ (S4, días 4–5)

- Regla 1 (datos universales): los datos básicos del paciente (nombre, CURP, alergias, antecedentes) son visibles para cualquier médico del Distrito con un encuentro activo. Sin restricción de establecimiento.
- Regla 2 (notas por establecimiento): una nota SOAP solo es visible para el médico que la redactó y para médicos del mismo establecimiento, excepto cuando existe una referencia con estado ACEPTADA o ATENDIDA.
- Regla 3 (acceso por especialidad): el médico receptor de una referencia solo puede ver las notas relevantes a la especialidad de la referencia, no el expediente completo.
- Regla 4 (auditoría sin excepción): toda lectura de expediente, nota o prescripción registra en auditoria_accesos con IP, módulo, función y nivel_severidad. El middleware de P1 (S1–S2) lo hace automáticamente; P3 debe asegurarse de no bypassarlo.
- Implementar como funciones de service: check_regla_1(id_paciente, id_usuario), check_regla_2(id_nota, id_usuario), check_regla_3(id_referencia, id_especialidad, id_usuario). Cada endpoint las llama antes de devolver datos.

 **Referencia:** Descripción de las 4 Reglas de Negocio: Doc6 §Diagrama 6 — flujo completo de control de acceso.

 **Referencia:** Middleware de auditoría (Regla 4): Doc2 §1.2 FastAPI — intercepta automáticamente todos los requests.

Módulo Referencias Médicas (S6)

- Implementar POST /referencias (emitir), PATCH /referencias/{id} (aceptar/rechazar), POST /referencias/{id}/contrarreferencia.
- Estados de la referencia: PENDIENTE → ACEPTADA → ATENDIDA → CONTRARREFERIDA (o RECHAZADA). Cada transición de estado se registra en auditoria_accesos.
- Al aceptar una referencia (estado → ACEPTADA), el service debe verificar que el establecimiento destino tiene la especialidad habilitada en establecimientos_especialidades (cruzar con el catálogo que administra P5).

- La contrarreferencia crea un registro nuevo en `referencias_medicas` con tipo `CONTRARREFERENCIA`, vinculado a la referencia original por `id_referencia_origen`.

 **Referencia:** Flujo completo de referencia entre establecimientos: Doc6 §Diagrama 5 — Sub-flujos A, B y C con todos los pasos.

 **Referencia:** Endpoints de referencias: Doc3 — buscar Módulo Referencias en el índice de la API.

Frontend de expediente (S6)

- Implementar `PacientesListPage.jsx`: tabla paginada con búsqueda por nombre, primer apellido y número de expediente. Usar el componente `Table` de `shadcn/ui` instalado por P2.
- Implementar `PacienteFichaPage.jsx`: datos demográficos, badge de barrera lingüística si `id_lengua_materna != Español (#D97706)`, lista de alergias con badge por severidad (Doc7 §1.3).
- Implementar `ExpedientePage.jsx`: tabs para Antecedentes (4 tipos), Alergias (ordenadas CRÍTICA primero), Inmunizaciones, Historial de encuentros con referencias activas.
- Conectar todo con el `AuthContext` de P2 y usar los endpoints de Doc3 §Módulos 2, 3 y 4.

 **Referencia:** Especificaciones de UI del expediente: Doc7 §4 — módulo Expediente.

 **Referencia:** Flujo de consulta completa: Doc6 §Diagrama 3 — el expediente se muestra antes del paso 1 del SOAP.

3.4 — Persona 4 — Backend SOAP, Consulta & Documentos

Semanas activas	S4–S5 (backend + frontend consulta)
Carga estimada	Alta — módulo más crítico del flujo clínico operativo

Rol en el proyecto

Persona 4 implementa el acto médico completo: el stepper de 5 pasos del SOAP, la firma electrónica de notas (NOM-151) y el PDF de nota firmada. También implementa solicitudes de estudio. Este módulo es el que los médicos usan en cada consulta y debe seguir estrictamente el flujo del Diagrama 3 (Doc6).

Módulo Encuentros Clínicos (S4, días 1–2)

- Implementar POST /encuentros, GET /encuentros, GET /encuentros/{id}, PATCH /encuentros/{id}/cerrar, GET /pacientes/{id}/encuentros según Doc3 §Módulo 5.
- El cierre de un encuentro (PATCH /cerrar) es IRREVERSIBLE. Requiere que al menos una nota SOAP esté firmada (verificar esta_firmada=TRUE en notas_medicas antes de cerrar).
- fecha_inicio y fecha_cierre se registran con TIMESTAMPTZ del servidor PostgreSQL (NOT NOW() del cliente), conforme a NOM-004 (Doc3 §Módulo 5, Reglas de negocio).
- Regla 4: la apertura y el cierre del encuentro deben registrarse en auditoria_accesos con nivel_severidad MEDIO. El middleware de P1 lo hace, pero verificar que el módulo y función se registran correctamente.

 **Referencia:** Flujo del encuentro: Doc6 §Diagrama 3 — Precondición: médico tiene encuentro activo con fecha_cierre = NULL.

 **Referencia:** Endpoints completos: Doc3 §Módulo 5 — request/response de apertura y cierre.

Módulo Signos Vitales (S4, días 2–3)

- Implementar POST /signos-vitales con validación de rangos fisiológicos: tensión sistólica 60–250 mmHg, diastólica 40–150 mmHg, temperatura 34–42 °C, saturación O2 70–100%, frecuencia cardíaca 30–220 lpm.
- El campo registrado_en es TIMESTAMPTZ del servidor (Doc6 §Diagrama 3, Paso 2 — trazabilidad NOM-004 garantizada). El cliente no puede enviarlo.
- GET /signos-vitales?id_encuentro={id} retorna los signos del encuentro activo — P3 creó la vista v_signos_encuentro que P4 puede usar en este endpoint.
- En el frontend, el paso 1 del stepper (Signos) está completo cuando todos los signos obligatorios tienen valor y están dentro de rango.

 **Referencia:** Flujo de registro de signos: Doc6 §Diagrama 3, Pasos 1 y 2 — actor Enfermería registra, sistema guarda timestamp.

Módulo Notas SOAP — backend (S4, días 3–5)

- Implementar POST /encuentros/{id}/notas (crear nota en borrador), PATCH /notas/{id} (actualizar mientras esta_firmada=FALSE), PATCH /notas/{id}/firmar según Doc3 §Módulo 6.

- La firma (PATCH /notas/{id}/firmar) activa el trigger tr_notes_protection en PostgreSQL, que bloquea cualquier UPDATE posterior a nivel de BD. El service solo necesita llamar el PATCH; el trigger lo hace inmutable automáticamente (ya definido en el modelo de BD).
- La firma incluye: flag esta_firmada=TRUE, firmado_en (TIMESTAMPTZ servidor), firmado_por (id_usuario), cedula_profesional (tomada del usuario_sistema en el momento de la firma) y hash SHA-256 del contenido de la nota.
- Implementar POST /notas/{id}/enmienda para correcciones post-firma. El error original permanece visible; la enmienda es un Addendum en notas_enmienda (Doc3 §Módulo 6, Reglas).
- Implementar GET /catalogos/cie10?q={termino} para búsqueda de diagnósticos con ILIKE en cat_cie10 — retornar máximo 20 resultados ordenados por relevancia.

 **Referencia:** Contrato completo de notas SOAP: Doc3 §Módulo 6 — incluyendo inmutabilidad y enmiendas.

 **Referencia:** Flujo de firma NOM-151: Doc6 §Diagrama 3, Pasos 9–12 — firma → hash → sello visual.

Generación de PDF de nota SOAP (S5, días 1–2)

- Implementar GET /notas/{id}/pdf con WeasyPrint para generar el PDF de nota SOAP firmada.
- El PDF debe incluir (NOM-004 y NOM-151): datos CLUES del establecimiento, nombre y cédula del médico, número de expediente, fecha y hora de la consulta en hora CST (convertir de UTC), contenido SOAP completo, hash SHA-256 del contenido y sello visual de inmutabilidad (#2D8653 verde, Doc7 §1.3).
- El PDF se genera on-demand y NO se persiste en Azure Blob Storage — se devuelve como response binario con Content-Type: application/pdf (decisión de arquitectura Doc6 §Capa 5).
- Usar la plantilla HTML con los estilos de Doc7 §1.1 (DM Sans, colores institucionales) para mantener coherencia visual.

 **Referencia:** Decisión de no persistir PDFs generados: Doc6 §Diagrama 1, Capa 5 — Blob Storage solo para PDFs externos.

 **Referencia:** Campos obligatorios del PDF: Doc6 §Diagrama 3, Paso 12 — sello visual post-firma.

Módulo Solicitudes de Estudio (S5, día 3)

- Implementar POST /solicitudes-estudio con campos: id_encuentro, tipo_estudio, urgente (boolean), indicacion_clinica, id_cie10_relacionado.
- GET /solicitudes-estudio?id_encuentro={id} retorna las solicitudes del encuentro, ordenadas por urgente DESC.
- El PDF de solicitud de laboratorio lo genera P5 (WeasyPrint), P4 solo implementa los endpoints backend y provee los datos.

 **Referencia:** Contrato del endpoint: Doc3 §Módulo — buscar solicitudes_estudio en la documentación de la API.

Frontend de consulta — Stepper 5 pasos (S5, días 3–5)

- Implementar ConsultaPage.jsx con stepper de 5 pasos: Paso 1 Signos Vitales, Paso 2 Subjetivo, Paso 3 Objetivo, Paso 4 Análisis (diagnóstico CIE-10), Paso 5 Plan + Firma.

- Cada paso guarda automáticamente un timestamp al avanzar (gap de NOTAS_PENDIENTES.md, Doc6 §Diagrama 3 Paso 7 — el sistema guarda timestamp al avanzar cada sección).
- El paso 4 incluye autocompletado de CIE-10 conectado a GET /catalogos/cie10?q= — mostrar código + descripción con Combobox de shadcn/ui.
- Si el paciente tiene alergia CRÍTICA, mostrar badge rojo prominente #DC2626 en el header de la consulta durante todo el stepper (Doc7 §1.3 — Alergia ALTA severidad).
- Post-firma: mostrar sello visual verde (#2D8653) con texto 'Nota firmada — Inmutable' + hash SHA-256 truncado a 8 caracteres (Doc7 §1.3 — Nota FIRMADA).
- Si id_lengua_materna del paciente != Español, mostrar callout ámbar #D97706 'Barrera lingüística detectada' (Doc7 §1.3 y Doc3 §Módulo 2).

 **Referencia:** Especificaciones completas del stepper SOAP: Doc7 §4 — Módulo Consulta.

 **Referencia:** Flujo paso a paso del acto médico: Doc6 §Diagrama 3 — todos los actores y estados.

 **Referencia:** Colores semánticos del stepper: Doc7 §1.3 — tabla completa de contextos clínicos.

3.5 — Persona 5 — Backend Prescripciones, Lab & Frontend Admin/Auditoría

Semanas activas	S5–S7 (backend + frontend admin y auditoría)
Carga estimada	Alta — cierra el ciclo clínico y habilita el control de seguridad

Rol en el proyecto

Persona 5 cierra el ciclo clínico con prescripciones y laboratorio, y entrega los frontends de administración y auditoría que hacen que el sistema sea operable desde el día 1. La alerta de alergia crítica al prescribir es un requisito de seguridad del paciente de alta prioridad.

Módulo Prescripciones + alerta de alergia (S5, días 1–3)

- Implementar POST /prescripciones con verificación activa de alergias: antes de guardar, buscar en la tabla alergias del paciente si alguna sustancia_alergica coincide (ILIKE) con el medicamento seleccionado de cat_medicamentos.
- Si hay coincidencia con severidad CRÍTICA: retornar HTTP 409 Conflict con el detalle de la alergia para que el frontend muestre la alerta. El médico debe confirmar explícitamente para continuar.
- Si la coincidencia es MODERADA o LEVE: retornar HTTP 200 con flag alergia_advertencia=true y detalle — el frontend muestra un warning no bloqueante.
- Implementar GET /prescripciones?id_encuentro={id}, PATCH /prescripciones/{id} (solo mientras la nota no esté firmada) y DELETE lógico.
- Implementar GET /catalogos/medicamentos?q={nombre} con búsqueda ILIKE en cat_medicamentos — retornar nombre genérico, presentación y concentración.

 **Referencia:** Lógica de alerta de alergia: Doc3 §Módulo — buscar prescripciones. La verificación es en el service, no en el trigger.

 **Referencia:** Colores de alerta en frontend: Doc7 §1.3 — CRÍTICA #DC2626 bloqueante, MODERADA #D97706 advertencia.

Módulo Resultados de Laboratorio + Azure Blob (S5, días 4–5)

- Implementar POST /resultados-laboratorio/upload: recibe el PDF externo (multipart/form-data), calcula SHA-256 del archivo, sube a Azure Blob Storage usando la cadena de conexión AZURE_STORAGE_CONN_STR de Doc4 §1.1.
- Generar SAS token con TTL de 15 minutos para el acceso temporal al archivo (BLOB_SAS_TTL_MIN=15, Doc4 §1.1). El frontend nunca recibe la URL directa de Blob, solo el SAS token.
- Registrar en auditoria_accesos el hash SHA-256 del PDF de laboratorio con nivel_severidad ALTO (NOM-151 — trazabilidad de documentos externos).
- GET /resultados-laboratorio?id_solicitud={id} retorna el resultado con un SAS token generado al momento del request (no almacenado).

 **Referencia:** Arquitectura de Azure Blob Storage: Doc6 §Diagrama 1, Capa 5 — Blob solo para archivos de origen externo.

 **Referencia:** Variables de Azure Blob: Doc4 §1.1 — AZURE_STORAGE_CONN_STR y BLOB_SAS_TTL_MIN.

PDFs con WeasyPrint — Receta, Solicitud y Referencia (S6, días 1–3)

- Implementar GET /prescripciones/{id_encuentro}/pdf — receta médica nominativa con WeasyPrint: nombre y cédula del médico, datos del paciente, listado de medicamentos con dosis e indicaciones, establecimiento y CLUES, timestamp CST.
- Implementar GET /solicitudes-estudio/{id}/pdf — solicitud de laboratorio: tipo de estudio, indicación clínica, diagnóstico CIE-10 relacionado, médico solicitante, urgente destacado visualmente.
- Implementar GET /referencias/{id}/pdf — resumen de referencia médica: motivo, diagnóstico, establecimiento destino, especialidad, nivel de urgencia y firma del médico emisor.
- Los 3 PDFs usan la paleta de Doc7 §1.1 (DM Sans, #1B4F8A azul institucional) y se generan on-demand sin persistir en Blob.

 **Referencia:** Datos clínicos obligatorios en PDFs: Doc3 §Módulo 6 — sello NOM-151 con cédula y timestamp.

 **Referencia:** Decisión de no persistir PDFs: Doc6 §Diagrama 1, Capa 5 — generados on-demand con WeasyPrint.

Frontend de Administración (S6, días 4–5)

- Implementar UsuariosAdminPage.jsx: tabla paginada de usuarios_sistema con búsqueda. Modal de creación/edición con campos: nombre, username, rol (select de los 6 roles de los seeds de P2), establecimiento activo, cédula profesional.
- Implementar EstablecimientosPage.jsx: CRUD de establecimientos con campos CLUES (único), nombre, nivel de atención, municipio (select de cat_municipios cargado por P2).
- Implementar EspecialidadesEstablecimientoPage.jsx: tabla de establecimientos_especialidades — habilitar/deshabilitar especialidades por unidad médica con toggle. Esta tabla es consultada por P3 al validar referencias.
- Implementar RolesPermisosPage.jsx: visualización de la matriz de permisos_rol (lectura, los permisos no se editan desde UI en esta versión).
- Usar los componentes Table, Dialog y Badge de shadcn/ui instalados por P2.

 **Referencia:** Entidades afectadas por admin: Doc1 §2.5 original — tablas usuarios_sistema, establecimientos, roles, permisos_rol.

 **Referencia:** Casos de uso del Administrador: Doc6 §Diagrama 2, Actor 4 — ADMINISTRADOR.

Frontend de Auditoría y Seguridad (S7)

- Implementar AuditoriaPage.jsx: tabla paginada de auditoria_accesos con filtros por tipo_evento, nivel_severidad, fecha_desde/hasta y id_usuario. Usar v_auditoria_estadistica de P3 para el panel de resumen (contadores por tipo).
- Implementar IncidentesPage.jsx: listado de incidentes_seguridad con ciclo de vida ABIERTO → EN_PROCESO → RESUELTO → CERRADO. Botones de transición de estado visibles según el estado actual. Solo visible para AUDITOR_SEGURIDAD y SUPERADMIN.
- Implementar SeguridadPage.jsx: visualización de sesiones_invalidas activas (blacklist de tokens) y estadísticas de v_auditoria_estadistica. Solo SUPERADMIN.
- El indicador de incidente CRÍTICO en el sidebar (#DC2626 parpadeante) se activa si hay algún incidente con nivel CRÍTICO sin resolver — implementar polling de 60s con useEffect.

 **Referencia:** Ciclo de vida de incidentes: Doc6 §Diagrama 6 — flujo completo ABIERTO → CERRADO.

 **Referencia:** Colores del panel de seguridad: Doc7 §1.3 — Incidente CRÍTICO #DC2626 parpadeante.

 **Referencia:** Roles con acceso al panel: Doc6 §Diagrama 2, Actores 5 y 6 — AUDITOR_SEGURIDAD y SUPERADMIN.

4. Plan de 9 Semanas — Vista Consolidada

El plan mantiene los mismos hitos del Doc1 original pero redistribuye las actividades de implementación. Los entregables de cada semana no cambian — lo que cambia es quién los hace.

Semana	P1 (Lead)	P2 (Compa)	P3 (Clínico)	P4 (SOAP)	P5 (Admin)	Entregable
S1	Repo, Docker, .env, scripts	Vite setup, layout, paleta, DM Sans	—	—	—	Repo ejecutable
S2	Revisión de PRs	AuthContext, Login 2FA, Seeds SQL, shadcn/ui	—	—	—	Login 2FA funcional
S3	PRs	Ajustes menores	Personas, Pacientes, CRUD base	—	—	API pacientes en Swagger
S4	PRs	—	Alergias, Antecedentes, 4 Reglas, Vistas SQL	Encuentros, Signos, SOAP backend	—	Expediente + Encuentro funcionales
S5	PRs	Ajustes UX (expediente #)	—	Firma, PDF nota, Solicitudes, Stepper frontend	Prescripciones + alerta alergia, Lab upload	SOAP firmado + PDF nota

Semana	P1 (Lead)	P2 (Compa)	P3 (Clínico)	P4 (SOAP)	P5 (Admin)	Entregable
S6	PRs	—	Referencias médicas + frontend expediente	—	3 PDFs + frontend admin CRUD	4 PDFs + Referencias
S7	Pruebas integración	—	—	—	Frontend auditoría + seguridad	Panel auditoría funcional
S8	Deploy Azure (config)	—	Apoya deploy	Bug fixes	Bug fixes	URL Azure con HTTPS
S9	Demo	Demo	Demo	Demo	Demo	Entrega final

5. Guía de Referencias Cruzadas para Uso con IA

💡 Cómo usar este documento con ChatGPT o Claude

Cada tarea de implementación incluye referencias a documentos técnicos específicos. Para obtener código funcional, adjunta este documento + el documento referenciado al LLM con el prompt: 'Implementa [tarea] siguiendo exactamente las especificaciones de las secciones referenciadas. Usa el stack de Doc2 y respeta las convenciones de nomenclatura de Doc1 §6.'

5.1 Mapa de Documentos de Referencia

Documento	Título	Cuándo usarlo
Doc1 §5–6	Convenciones	Siempre — nombres de archivos, funciones, commits, branches
Doc2 §1	Arquitectura & Stack	Para justificar decisiones técnicas, resolver dudas de stack (FastAPI, SQLAlchemy, WeasyPrint, pyotp)
Doc3	API REST	Para implementar cualquier endpoint — request/response, auth requerida, tablas afectadas
Doc4 §1–2	Infraestructura	Para variables de entorno (.env), docker-compose, Dockerfile, Azure deploy
Doc5	Seeds & Catálogos	Para generar seeds SQL, entender fuentes oficiales (INEGI, DGIS, INALI, SSA)
Doc6	Diagramas & Flujos	Para implementar lógica de negocio — Diagrama 3 (SOAP), D4 (Login), D5 (Referencias), D6 (Incidentes)
Doc7	UI/UX	Para implementar cualquier componente de frontend — colores exactos, dimensiones, tipografía, comportamiento de badges

5.2 Prompts sugeridos por persona

Ejemplos de prompts efectivos para acelerar la implementación:

- **P2 — Seeds CIE-10:** "Implementa el script Python de Doc5 §2.3 para convertir el archivo ITER del INEGI a SQL de localidades. El script está completo en ese documento, solo necesito ejecutarlo contra el archivo real."
- **P2 — AuthContext:** "Implementa el AuthContext de React según Doc2 §1.1 (JWT en memoria, NO localStorage) y el flujo de login de Doc3 §Módulo 1. Sigue las convenciones de carpetas de Doc1 §6.2."

- **P3 — Reglas de negocio:** "Implementa las 4 Reglas de Negocio de MedIA como funciones de service en FastAPI/SQLAlchemy. El flujo completo está en Doc6 §Diagrama 6. Las tablas afectadas están en Doc3 §Módulo 3 y 4."
- **P4 — Firma de nota:** "Implementa PATCH /notas/{id}/firmar en FastAPI según Doc3 §Módulo 6. Debe calcular SHA-256 del contenido, activar el trigger tr_notes_protection y seguir el flujo del Doc6 §Diagrama 3, Pasos 9–12."
- **P5 — Alerta de alergia:** "Implementa el service de prescripciones con verificación de alergias críticas según Doc3 §Módulo — Prescripciones. Si hay coincidencia CRÍTICA devuelve HTTP 409; MODERADA devuelve 200 con alergia_advertencia=true."

6. Tabla de Módulos, Tablas BD y Responsables

Módulo	Tablas principales	Responsable	Semana
Auth (JWT+2FA)	usuarios_sistema, sesiones_invalidas, auditoria_accesos	P1 — ya diseñado en Doc3 §Módulo 1; P1 solo aprueba la implementación de P2 (AuthContext)	S2
Seeds & Catálogos	cat_estados, cat_municipios, cat_localidades, cat_cie10, cat_medicamentos, roles, permisos_rol	P2 — Doc5 tiene SQL y scripts completos	S1–S2
Frontend base	—	P2 — Sidebar, TopBar, AuthContext, Login, Dashboard	S1–S2
Personas	personas, cat_estados, cat_municipios, cat_localidades, cat_lenguas_indigenas	P3	S3
Pacientes	pacientes, pacientes_tutores_representantes	P3	S3
Expediente	alergias, antecedentes_*, inmunizaciones, historial_cambios	P3	S4
4 Reglas Negocio	services/ (lógica, no tablas propias)	P3	S4
Referencias SRC	referencias_medicas, establecimientos_especialidades	P3	S6
Frontend expediente	—	P3 — Listado, ficha, expediente completo	S6
Encuentros	encuentros_clinicos, auditoria_accesos	P4	S4
Signos Vitales	signos_vitales	P4	S4
Notas SOAP	notas_medicas, notas_soap_detalle, notas_enmienda, diagnosticos_encuentro	P4	S4–S5

Módulo	Tablas principales	Responsable	Semana
PDF nota SOAP	—	P4 — WeasyPrint on-demand	S5
Solicitudes estudio	solicitudes_estudio	P4	S5
Frontend consulta	—	P4 — Stepper 5 pasos SOAP	S5
Prescripciones	prescripciones, cat_medicamentos, alergias (lectura)	P5 — con alerta de alergia crítica	S5
Lab + Azure Blob	resultados_laboratorio, auditoria_accesos	P5 — upload + SHA-256 + SAS token	S5
3 PDFs	—	P5 — receta, solicitud lab, referencia	S6
Frontend admin	usuarios_sistema, establecimientos, roles, permisos_rol, establecimientos_especialidades	P5	S6
Frontend auditoría	auditoria_accesos, incidentes_seguridad, v_auditoria_estadistica	P5	S7
Deploy Azure	—	P1 (configuración) + P3 (apoya backend)	S8

✓ Resumen final

P1 (Lead) coordina y hace el setup inicial — no más doble o triple jornada. P2 establece la base del frontend y los seeds, trabajo de alta prioridad del que depende todo el equipo. P3, P4 y P5 llevan cargas equivalentes en el backend y frontend clínico, y trabajan en paralelo mediante ramas de feature independientes con integración continua en develop (ver §7). Todas las tareas tienen referencia exacta al documento técnico que las especifica, para que el equipo pueda trabajar de forma autónoma.

7. Flujo de Trabajo Git Paralelo — P3, P4 y P5

P3, P4 y P5 trabajan en paralelo desde la Semana 3 sin bloquearse entre sí. Cada persona mantiene su propia rama de feature, hace push cuando termina una tarea, y

las integraciones ocurren en develop mediante Pull Requests aprobados por P1. El objetivo es reducir el cronograma de 6 a 3–4 semanas de desarrollo efectivo.

7.1 — Estructura de ramas

El repositorio maneja tres niveles de ramas:

main — código de producción. Solo P1 hace merge aquí, desde develop, previo al deploy de S8. Requiere PR aprobado.

develop — rama de integración compartida. Todos los PRs de feature apuntan aquí. Es la fuente de verdad del trabajo integrado del equipo.

feature/pX-nombre-modulo — una rama por módulo por persona. Ejemplos: feature/p3-pacientes-expediente, feature/p4-encuentros-soap, feature/p5-prescripciones-admin.

7.2 — Ciclo de trabajo por módulo

Cada integrante sigue este ciclo para cada módulo que implementa:

1. Crear rama desde develop: `git checkout develop && git pull origin develop && git checkout -b feature/pX-nombre`
2. Desarrollar el módulo. Hacer commits frecuentes con la convención feat(módulo): descripción (Doc1 §5).
3. Al terminar el módulo (o al final del día): `git push origin feature/pX-nombre` y abrir Pull Request hacia develop en GitHub.
4. P1 revisa y aprueba el PR. Una vez mergeado a develop, los demás hacen `git pull origin develop` en sus ramas para incorporar los cambios.
5. Continuar con el siguiente módulo repitiendo el ciclo desde el paso 1, ya con los cambios del equipo integrados.

7.3 — Dependencias entre P3, P4 y P5

La mayoría de los módulos son independientes. Las únicas dependencias reales son las siguientes:

P4 depende de P3 (parcialmente): P4 necesita los endpoints de Pacientes y los schemas Pydantic de P3 para conectar la lógica de encuentros con el expediente. P4 puede arrancar en paralelo implementando modelos de BD, schemas y lógica de encuentros; solo necesita esperar el merge de P3 a develop para conectar la verificación de acceso (`check_regla_1/2`). P3 debe hacer merge de su módulo de Pacientes a develop antes del final de S3.

P5 es completamente independiente de P3 y P4: Prescripciones, Lab, Azure Blob, PDFs y el frontend de admin/auditoría no tienen dependencias de los módulos de P3 ni P4 más allá de la tabla alergias (que P3 define en el modelo). P5 puede arrancar en S3 con los modelos de prescripciones sin esperar ninguna integración.

Coordinación mínima requerida en S3 (primer día): P3 publica los schemas Pydantic de Paciente y las firmas de check_regla_1/2/3 en un archivo services/interfaces.py que sube a develop aunque la implementación no esté completa. Esto permite a P4 programar contra la interfaz sin bloqueos.

7.4 — Cronograma comprimido: de 6 a 4 semanas

Gracias al trabajo en paralelo, el desarrollo efectivo se reduce de 6 semanas secuenciales a 4 semanas con el siguiente esquema de solapamiento:

Semana	P3 (Backend Clínico)	P4 (SOAP & Consulta)	P5 (Prescripciones & Admin)
S3	Personas, Pacientes + publica interfaces a develop	Modelos encuentros, schemas, lógica base (sin conectar a P3 aún)	Modelos prescripciones, tablas lab — totalmente independiente
S4	Alergias, antecedentes, 4 Reglas, vistas SQL → merge a develop	pull develop → conecta check_regla_1/2, Encuentros completos, Signos, SOAP	Prescripciones + alerta alergia, Lab + Azure Blob
S5	Referencias médicas	Firma, PDF nota, Solicitudes, Stepper frontend 5 pasos	3 PDFs WeasyPrint, Frontend Admin
S6	Frontend expediente completo	Ajustes + bug fixes	Frontend Auditoría + Seguridad

7.5 — Reglas de integración en develop

Nunca hacer push directo a develop ni a main. Todo cambio entra por PR.

Antes de abrir un PR, hacer git rebase origin/develop en la rama de feature para que el merge sea limpio y sin conflictos.

Cada PR debe incluir: descripción del módulo implementado, endpoints añadidos (si aplica) y cómo probarlo localmente (docker compose up + migrate.sh + seed.sh).

Si dos personas modifican el mismo archivo (ej. models.py), el segundo en hacer merge debe resolver conflictos antes de pedir revisión. P1 no resuelve conflictos de implementación.

Hacer pull de develop al inicio de cada día de trabajo: git pull origin develop --rebase. Esto mantiene las ramas individuales actualizadas con el trabajo integrado del resto.